Distributed Multi-Model Analytics for E-commerce Data

Student: UBANJE Albert
Student ID: 101402
Course: Big Data Analytics (AUCA) — Final Project
Date: 11-11-2026
GitHub: https://github.com/JunoSoft/UBANJE_Albert_101402_BigDataAnalyticsFinalProject

Executive Summary

This report describes a multi-model analytics stack built on synthetic e-commerce data (users, products, categories, browsing sessions and transactions over a 90-day window). MongoDB stores nested business entities, HBase stores high-volume time-oriented session activity, and Apache Spark performs batch cleaning, recommendation indicators, SQL analytics and cross-entity workflows.

Headline findings (DEMO run, 40,000 sessions):

- Only **18.8%** of sessions end in a purchase; the steepest losses occur from product view to cart and again between cart, checkout and purchase.
- Checkout completion is the weakest step: only **43.4%** of checkout sessions convert to a purchase.
- Platinum-tier customers spend on average about **3x** more than bronze-tier customers (about $15.2k vs. $4.8k).
- The top revenue category generates roughly **$1.51M** in line-item revenue and should receive assortment and promotion priority.

1. System Architecture Overview

The project implements a multi-model analytics stack for synthetic e-commerce data. The design intentionally assigns each technology the workload that matches its strengths, rather than forcing one database to solve every access pattern.

Component	Role	Data focus
Dataset generator	Creates interconnected JSON entities over a 90-day window	Source-of-truth files
MongoDB	Document store for nested, query-flexible business entities	Products, users, categories, transactions
HBase	Wide-column store for high-volume time-series and sparse events	User session streams, product daily metrics
Apache Spark	Distributed batch processing and cross-store style joins	Cleaning, recommendations, SQL, integrated KPIs
Visualization layer	Communicates findings	Static charts for report and presentation

Data flow: JSON files feed (a) the MongoDB load and aggregations, (b) the HBase load and scans, and (c) Spark DataFrames/SQL over the same files (and notionally over Mongo/HBase extracts). Spark then produces the integrated funnel and CLV analyses, which are turned into charts and business insights.

2. Data Modeling Decisions

2.1 MongoDB document model

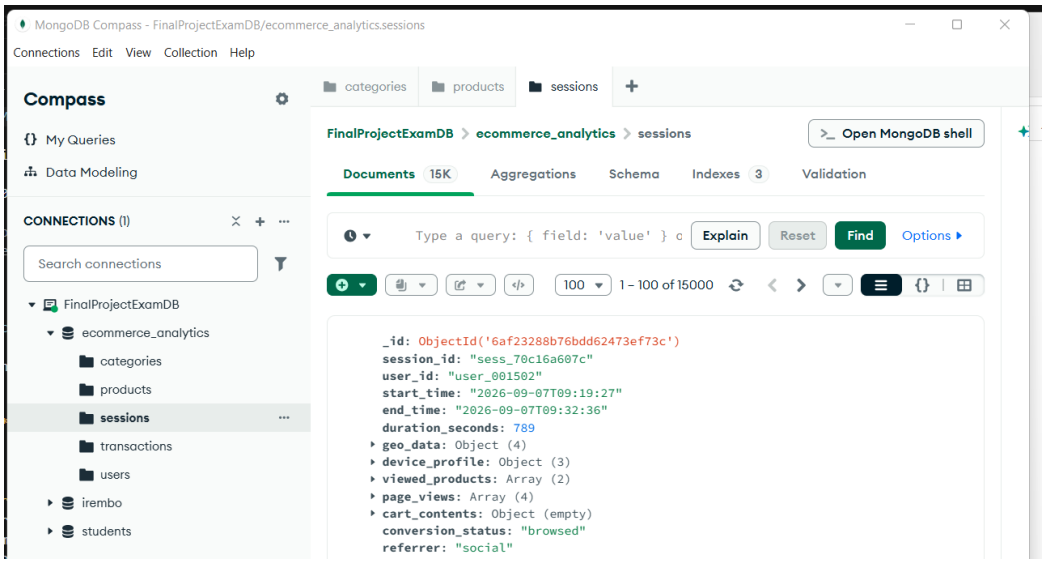
MongoDB was chosen for entities that are **naturally nested** and frequently retrieved as a whole document.

- **categories:** hierarchy embedded as `subcategories[]`, avoiding multi-collection joins for catalog browsing.
- **products:** catalog fields plus embedded `price_history[]`; references `category_id` / `subcategory_id`.
- **transactions:** order header plus embedded `items[]` (unit price, quantity, subtotal), so one read returns a complete purchase.
- **users:** profile plus a denormalized `purchase_summary` (order_count, lifetime_spend, average order value) computed at load time to accelerate segmentation.

Indexes prioritize analytical access paths: `product_id`, `category_id`, `user_id`, `timestamp`, `items.product_id` and `purchase_summary.order_count`.

Aggregation pipelines implemented:

- Revenue by category (unwind items, lookup products/categories, group).
- User segmentation by purchase frequency (non_buyer / occasional / regular / power_buyer).
- Top-selling products by units and revenue.



MongoDB Compass: the sessions collection (15,000 documents) loaded with embedded device_profile, page_views and cart_contents.

2.2 HBase wide-column model

HBase was chosen for **append-heavy, time-oriented, sparse** activity data.

- **Table user_sessions:** row key `user_id#reverse_timestamp#session_id`; column families `info` (metadata), `geo` and `events` (page_view JSON columns). This enables prefix scans for "recent activity of user X".
- **Table product_metrics:** row key `product_id#yyyy-MM-dd`; family `stats` (views, cart_adds, purchases). This enables efficient product time-series retrieval for conversion analysis.

Example HBase shell commands for this schema (schema creation and a per-user prefix scan):

```
create 'user_sessions', 'info', 'geo', 'events'
create 'product_metrics', 'stats'
scan 'user_sessions', {STARTROW => 'user_000042#', STOPROW => 'user_000042$', LIMIT => 5}
```

2.3 MongoDB vs. HBase placement

Data / query	Store	Justification
Product catalog, flexible filters	MongoDB	Rich documents and secondary indexes
Orders with line items	MongoDB	Embedded arrays and aggregation framework
User profile + purchase summary	MongoDB	Document reads for CRM-style queries
Millions of page views / session events	HBase	Wide rows, sparse columns, sequential scans
Per-user activity stream	HBase	Row-key prefix design
Per-product daily counters	HBase	Time-series keys

A representative subset is loaded into each store (permitted by the assignment) while Spark processes the broader JSON corpus.

3. Spark Processing Pipelines

3.1 Batch cleaning and normalization

PySpark DataFrames ingest multiline JSON and then:

- cast numeric and timestamp fields,
- lowercase status fields,
- drop null keys and duplicate IDs,
- write cleaned Parquet under `outputs/results/cleaned/`.

This creates a reproducible, analytics-ready layer and demonstrates Spark's value for bulk ETL over many session files.

3.2 Recommendation indicators

We compute market-basket style signals:

- **Also bought:** pairwise co-occurrence of products inside the same transaction.
- **Viewed together:** pairwise co-occurrence inside the same session's `viewed_products`.

These indicators are simple, explainable and scalable with Spark's distributed group-bys, and are suitable as inputs to later collaborative-filtering models.

3.3 Spark SQL analytics

Temporary views simulate multi-store querying without mandatory connector setup (connector setup is optional and time-consuming per the project brief): `mongo_users`, `mongo_products`, `mongo_transactions` and `hbase_sessions`. SQL examples include revenue by payment method and month, category revenue joins, and engagement-versus-spend profiles joining user documents with session aggregates.

Optimization notes: `local[*]` master for development; Parquet materialization after cleaning; filter early; explode arrays only when needed; `approxQuantile` for tier thresholds instead of full sorts where possible.

4. Integrated Analytics Workflows

4.1 Funnel conversion analysis

Business question: where do users drop off between browse, product view, cart, checkout and purchase?

Sources: session page views (HBase model) and transactions linked by `session_id` (MongoDB model), joined in Spark.

Observed DEMO results (40,000 sessions):

Stage	Sessions	Step conversion	Sessions lost at step
Browse	40,000	—	—
Product view	39,503	98.8%	497
Cart	27,151	68.7%	12,352
Checkout	17,323	63.8%	9,828
Purchase	7,515	43.4%	9,808

Almost every session reaches a product page, but the funnel loses the most sessions between **product view and cart** (12,352 sessions). Cart-to-checkout (9,828) and checkout-to-purchase (9,808) losses are almost as large, and checkout-to-purchase has the lowest step conversion (43.4%). Overall, 18.8% of sessions end in a purchase. Priorities are therefore product-page persuasion (reviews, clear pricing) and, above all, checkout friction: payment UX and trust signals.

4.2 Customer Lifetime Value (CLV) estimation

Business question: which customers are high value when combining monetary spend with engagement?

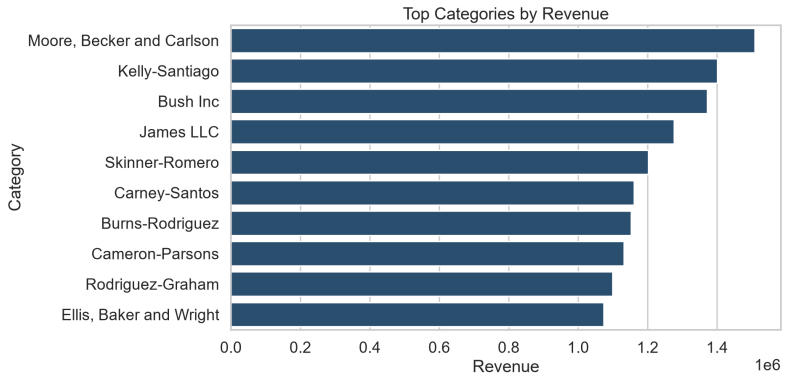
Score (explainable):

```
CLV = lifetime_spend + 0.5 x sessions + 5 x converted_sessions
```

Users are tiered (bronze, silver, gold, platinum) using empirical quantiles. This demonstrates cross-system enrichment: Mongo profiles and transactions, HBase engagement and Spark scoring.

5. Key Findings and Visualizations

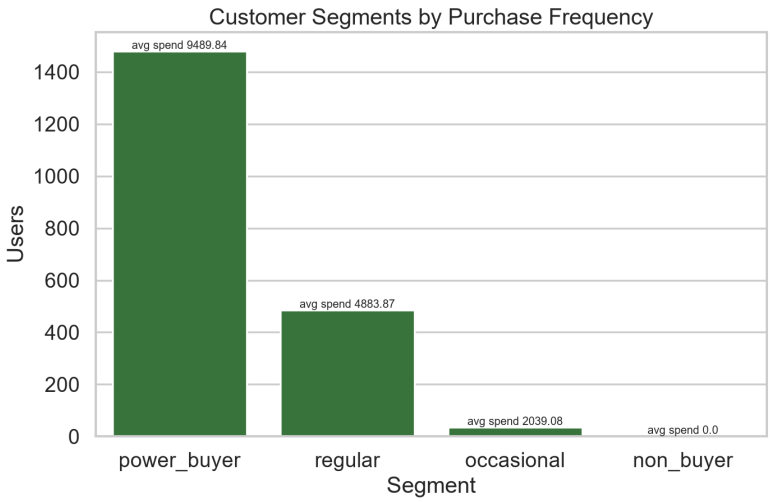
5.1 Top categories by revenue



Top 10 categories by line-item revenue (axis in units of 1e6 USD).

The leading category, **Moore, Becker and Carlson** (cat_007), earns about $1.51M in line-item revenue, followed by Kelly-Santiago and Bush Inc (both above $1.35M). Revenue is fairly evenly spread: the tenth category still exceeds $1.0M, so assortment and promotion priority should start with the top three while monitoring the long tail.

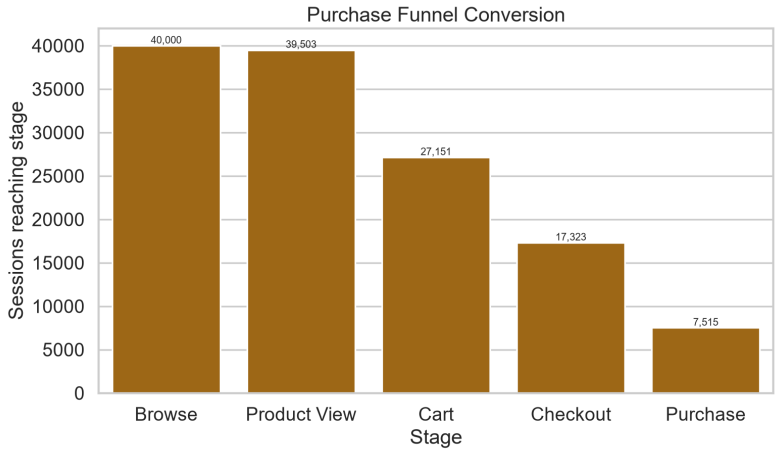
5.2 Customer segments by purchase frequency



Users per purchase-frequency segment, annotated with average spend.

Most users fall into the **power_buyer** segment (about 1,480 users, average spend $9,489.84), followed by **regular** (about 480 users, $4,883.87) and a very small **occasional** group ($2,039.08). Non-buyers are negligible. Average spend rises steeply with purchase frequency, so retention and repeat-purchase programs are the most direct way to increase revenue. The heavy concentration in power_buyer also suggests the segment thresholds are low for this synthetic dataset and should be recalibrated for real data.

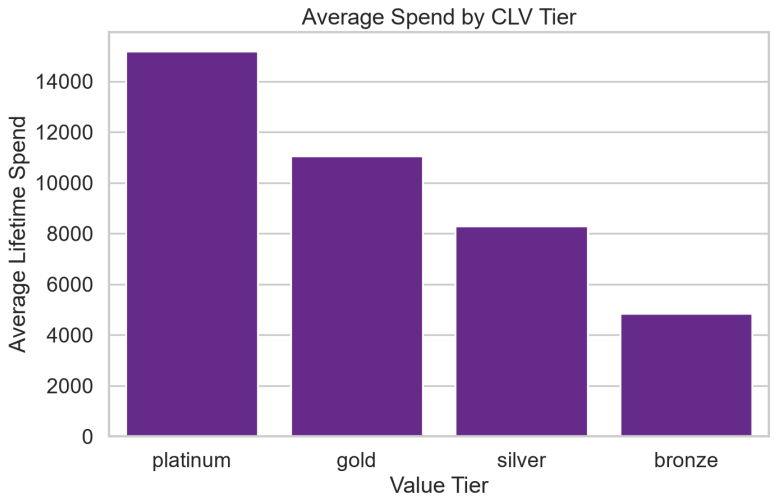
5.3 Purchase funnel



Sessions reaching each stage of the purchase funnel.

Sessions fall from 40,000 at browse to 7,515 purchases (about 18.8%). See Section 4.1 for step-level conversion and recommended actions.

5.4 Average spend by CLV tier



Average lifetime spend by CLV value tier.

Platinum users (n ≈ 219) average about **$15.2k** in lifetime spend, against roughly $11k for gold, $8.3k for silver and $4.8k for bronze. Loyalty budget and retention offers should concentrate on the platinum and gold tiers.

5.5 Summary of business insights

- **Assortment:** prioritize the top revenue categories (led by cat_007, about $1.51M) for promotion and stock depth.
- **Conversion:** improve product-to-cart and checkout completion; checkout-to-purchase conversion (43.4%) is the weakest step.
- **Loyalty:** concentrate retention spend on platinum and gold customers, who spend about 2 to 3 times more than bronze.
- **Cross-sell:** co-purchase pairs (e.g., prod_00053 and prod_00760) support recommendations on product and cart pages.

6. Technology Selection Justification

Analytical task	Best fit	Why
Nested order / catalog queries	MongoDB	Document model and aggregation pipeline
User activity time-range retrieval	HBase	Row-key scans on time-series keys
Multi-file cleaning and co-occurrence	Spark	Distributed CPU over large JSON/Parquet
Cross-entity KPI (funnel, CLV)	Spark (+ both stores)	Joins and windowed/quantile logic at scale

No single data model is optimal for all access patterns; the multi-model approach reduces awkward schemas and heavy application-side joins.

7. Scalability Considerations

- **MongoDB:** shard by `user_id` / `product_id`; keep the working set indexed; pre-aggregate summaries for dashboards.
- **HBase:** region splits on well-designed keys; salting if hot users dominate; TTL for raw events.
- **Spark:** move from `local[*]` to a YARN/Kubernetes cluster; partition Parquet by date; broadcast small dimensions (categories).
- **Ingestion:** FULL mode (2M sessions) should stream or chunk into HBase; DEMO mode validates logic quickly.

Bottlenecks are expected at session JSON size and HBase bulk writes; Parquet and batch puts mitigate them.

8. Limitations and Future Work

Limitations

- Synthetic data (Faker) lacks real seasonality and marketing shocks.
- HBase and Mongo loads use representative subsets for practicality.
- The CLV model is heuristic, not a probabilistic survival/CLV model.
- Spark connectors to live Mongo/HBase were simulated via DataFrame views.

Future work

- Structured Streaming for near-real-time funnel metrics.
- ALS collaborative filtering on Spark MLlib.
- Live Spark–Mongo and Spark–HBase connectors.
- Interactive dashboard (Plotly Dash / Power BI) on gold-layer Parquet.

9. Methodology Summary

1. Generate interconnected e-commerce JSON (DEMO or FULL).
2. Model and load MongoDB documents; run aggregation pipelines.
3. Design HBase keys and families; load sessions and product metrics; run prefix-scan queries.
4. Clean with Spark; compute recommendation pairs; run Spark SQL.
5. Integrate funnel and CLV across stores conceptually via Spark.
6. Visualize and interpret business actions.

All source code, scripts and reproduction steps are provided in the project GitHub repository and documented in its `README.md`:
https://github.com/JunoSoft/UBANJE_Albert_101402_BigDataAnalyticsFinalProject